

WEEK-3 [A*]

code:

```
import heapq

def misplaced_tiles(board, goal):
    """Counts how many tiles (excluding the blank 0 slot) are in the
    wrong position."""
    count = 0
    for i in range(9):
        if board[i] != 0 and board[i] != goal[i]:
            count += 1
    return count

def a_star(initial, goal):
    counter = 0
    h_initial = misplaced_tiles(initial, goal)
    queue = [(h_initial, 0, counter, initial, [initial])]
    visited = set()

    while queue:
        f, g, _, board, path = heapq.heappop(queue)

        if board == goal:
            return path

        if board in visited:
            continue
        visited.add(board)

        b = list(board)
        idx = b.index(0)
        r, c = idx // 3, idx % 3

        for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
            nr, nc = r + dr, c + dc
            if 0 <= nr < 3 and 0 <= nc < 3:
                n_idx = nr * 3 + nc
                b[idx], b[n_idx] = b[n_idx], b[idx]
                nxt = tuple(b)

                if nxt not in visited:
                    counter += 1
```

```

        f_next = g + 1 + misplaced_tiles(nxt, goal)
        heapq.heappush(queue, (f_next, g + 1, counter, nxt,
path + [nxt]))

        b[idx], b[n_idx] = b[n_idx], b[idx]

    return None

def print_matrix(board):
    for i in range(0, 9, 3):
        print(f" {board[i]} {board[i+1]} {board[i+2]} ")
    print()

initial = (2, 8, 3, 1, 6, 4, 7, 0, 5)
goal     = (1, 2, 3, 8, 0, 4, 7, 6, 5)

path = a_star(initial, goal)
if path:
    print(f"Initial State: {initial}")
    print(f"Final State: {goal}")
    print(f"Success! Found in {len(path) - 1} moves.\n")

    for step, board in enumerate(path):
        if step == 0:
            print("Initial Board:")
        else:
            print(f"Move {step}:")
        print_matrix(board)

```

OUTPUT :

Initial State: (2, 8, 3, 1, 6, 4, 7, 0, 5)
Final State: (1, 2, 3, 8, 0, 4, 7, 6, 5)
Success! Found in 5 moves.

Initial Board:

2	8	3
1	6	4
7	0	5

Move 1:

2	8	3
1	0	4
7	6	5

Move 2:

2	0	3
1	8	4
7	6	5

Move 3:

0	2	3
1	8	4
7	6	5

Move 4:

1	2	3
0	8	4
7	6	5

Move 5:

1	2	3
8	0	4
7	6	5